

STATE DEPENDENT RICATTI EQUATION BASED ATTITUDE CONTROL USING MODIFIED RODRIGUEZ PARAMETERS

Mugdha Meda
ME21B119

Course Project
Dynamics and Control of Spacecrafts

Guide
Prof. Luoyi Tao

OBJECTIVE

The aim of this project is to implement an application of the State Dependent Riccati Equation (SDRE) method to satellite attitude control where the satellite kinematics is modeled by Modified Rodriguez Parameters (MRP). The SDRE methodology is applicable on special forms of non-linear systems. Point wise solutions are interpolated with respect to the operating conditions obtained from the MATLAB algorithms which are derived from the positive definite solutions of the SDRE. This only provides local stability, global stability analysis is difficult by this methodology. The resultant attitude controllers outside the selected operating conditions for interpolation are suboptimal. The performance of the designs is examined by simulations on MATLAB. The simulation results show that, the designed attitude controllers working satisfactorily.

The goal is to re-create learnings from papers by extrapolating given information, exploring analytical methods like Rodriguez Parameters to describe attitude and the Ricatti Equation-important for LQR control.

1. INTRODUCTION TO RODRIGUEZ PARAMETERS

Quaternions have 4 parameters. With modified Rodrigues, a new method with 3 parameters is created by dividing the vectorial 3 parameters with 1 scalar. Modified Rodriguez parameters (MRPs) can be expressed in terms of Euler axis and angle by:

$$p = \hat{e} \tan\left(\frac{\theta}{4}\right)$$

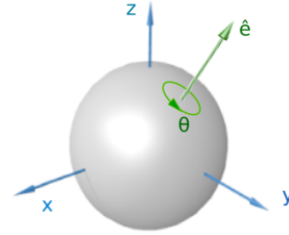


FIGURE 1. Euler axis $\hat{e}$ and angle θ .

For Quaternions to MRPs:

$$p_{x,y,z} = \frac{q_{i,j,k}}{1 + q_r}$$

Where $q_{i,j,k}$ is the vector part and q_r is the scalar part of a unit quaternion. We are able to avoid the singularities of Euler angles and provide a straightforward and efficient method for small-angle maneuvers. MRPs have a singularity when the rotation approaches 360, but this limitation does not produce a problem for practical ranges of rotations. The MRP is a three dimensional representation of the form:

$$\sigma = [\sigma_1 \ \sigma_2 \ \sigma_3]^T \in R^3$$

The kinematics of this representation: $\dot{\sigma} = G(\sigma)\omega$

The above kinematics represents the orientation of a three dimensional object with respect to a specific coordinate frame where the angular velocity vector $\omega \in R^3$ is also defined relative to the same coordinate frame. The matrix $G(\sigma)$ is invertible and real, defined as:

$$\mathbf{G}(\sigma) = \frac{1}{2} \left(\mathbf{I}_{3 \times 3} - \frac{\sigma^T \sigma}{2} \mathbf{I}_{3 \times 3} + S(\sigma) + \sigma \sigma^T \right) \quad (1)$$

And the notation $S(\sigma) \in R^{3 \times 3}$ is the skew-symmetric cross operator:

$$S(\sigma) = \begin{bmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & v_1 \\ -v_2 & v_1 & 0 \end{bmatrix}$$

The Rotation matrix is given by $R(\sigma) \in R^{3 \times 3}$:

$$R(\sigma) = \mathbf{I}_{3 \times 3} + \frac{8}{(1 + \sigma^T \sigma)^2} S(\sigma)^2 + 4 \frac{1 - \sigma^T \sigma}{(1 + \sigma^T \sigma)^2} S(\sigma)$$

2. SATELLITE MODELING

The coordinate frames and their notations that are used throughout the report correspond to:

1. Inertial Reference Frame:

The primary reference coordinate system whose origin is fixed at the center of the earth (Earth Centered Inertial or ECI). The z-axis coincides with the earth's spinning axis and points towards the north pole. The x-axis directed towards the vernal equinox and y-axis completes the right hand rule. It is denoted by i .

2. Orbit Referenced Frame:

The frame's origin is the center of mass of the satellite. Its z-axis points towards the center of the earth and x-axis points in the travelling direction of the satellite. Lastly, the y-axis completes the right hand rule. Satellite rotations around those axes are sometimes called as roll, pitch and yaw rotations respectively. This frame is denoted by o .

3. Body Fixed Reference Frame:

This coordinate system is fixed on the satellite's body and is generally selected to coincide with the principal axes of inertia. It is denoted by b .

3. ATTITUDE KINEMATICS AND DYNAMICS

3.1 Kinematics

The kinematics equation when satellite attitude is defined relative to the orbit frame can be rewritten as:

$$\dot{\sigma}_e = \mathbf{G}(\sigma_e) \omega_{ob}^b$$

In the above σ_e is the attitude of the satellite relative to the orbital frame and ω_{ob}^b is the angular velocity of the body relative to the orbital frame. The latter is defined mathematically as:

$$\omega_{ob}^b = \omega_{ib}^b - R(\sigma_e) \omega_{io}^o$$

Where ω_{ib}^b is the angular velocity of the satellite body relative to the ECI and ω_{ob}^b is the orbital velocity of the satellite. The rotation matrix $R(\sigma_e)$ is used to transform the orbital velocity to the satellite body frame. If the orbital frame is considered as the desired reference frame, this leads to a tracking error problem since the relative attitude of the satellite reduces to the attitude error with respect to the orbital reference frame. We can reduce the angular velocity term ω_{ob}^b as ω_e in order to stress that it is an angular velocity error. The re-written kinematics equation:

$$\dot{\sigma}_e = G(\sigma_e) \omega_e$$

3.2 Dynamics

For a rigid body satellite, the dynamics can be derived from the Newton's laws as in class. The resultant model in terms of the body angular velocity by Euler's Equation is the nonlinear differential equation:

$$\dot{\omega}_{ib}^b = -I^{-1} S(\omega_{ib}^b) I \omega_{ib}^b + I^{-1} \tau_a$$

where $\tau_a \in R^3$ is the actuator torque input and $I \in R^{3 \times 3}$ is the symmetric moment of inertia matrix shown as:

$$I = \begin{bmatrix} I_x & I_{xy} & I_{xz} \\ I_{xy} & I_y & I_{yz} \\ I_{xz} & I_{yz} & I_z \end{bmatrix}$$

Therefore the dynamics in terms of angular velocity error ω_e is:

$$\dot{\omega}_e = I^{-1}S(\omega_e + R(\sigma_e)\omega_{io}^o)[\omega_e + R(\sigma_e)\omega_{io}^o] - \dot{R}(\sigma_e)\omega_{io}^o + I^{-1}\tau_a$$

$$\dot{R}(\sigma_e) = S(\omega_e)R(\sigma_e)$$

For the sake of simplicity:

$$\dot{\omega}_e = -I^{-1}S(\omega_e)I\omega_e + I^{-1}u$$

Where, $u = -S(R(\sigma_e)\omega_{io}^o)I\omega_e - S(\omega_e)IR(\sigma_e)\omega_{io}^o$

$$-S(R(\sigma_e)\omega_{io}^o)IR(\sigma_e)\omega_{io}^o - IS(\omega_e)R(\sigma_e)\omega_{io}^o + \tau_a$$

4. STATE DEPENDENT RICCATI EQUATION (SDRE)

For a Linear State Space Time-Invariant Model-
 $\dot{x} = \mathbf{A}(x)x + \mathbf{B}(x)u$, where $\mathbf{A}(x) \in R^{n \times n}$ and $\mathbf{B}(x) \in R^{n \times n}$,
if we desire to minimize the state dependent quadratic cost
function (an LQR Problem):

$$J = \int_0^\infty \{ \mathbf{x}^T \mathbf{Q}(\mathbf{x})\mathbf{x} + \mathbf{u}^T \mathbf{R}(\mathbf{x})\mathbf{u} \} dt$$

For a control law: $u = -K(x)x$,
with $K(x) = -R^{-1}(x)B^T(x)P(x)$, where $P(x) \in R^{n \times n}$ must
satisfy:

$$A^T(x)P(x) + P(x)A(x) - P(x)B(x)R^{-1}(x)B^T(x)P(x) = -Q(x),$$

where $Q(x)$, $R(x) \in R^{n \times n}$ are Positive Definite and
symmetric. If a $P(x)$ is found such that it is Positive
Definite and symmetric, then the closed loop formed by
using the control law $u = -K(x)x$ is **point wise stable** at
each operating point $x = x_o$.

5. SDRE BASED ATTITUDE CONTROL

The final satellite attitude kinematics and dynamics
governing equations obtained:

$$\dot{\sigma}_e = G(\sigma_e)\omega_e$$

$$\dot{\omega}_e = -I^{-1}S(\omega_e)I\omega_e + I^{-1}u$$

Converting this to a linearized model about the operating
point chosen:

$$\begin{bmatrix} \dot{\sigma}_e \\ \dot{\omega}_e \end{bmatrix} = \begin{bmatrix} 0 & G(\sigma_e) \\ 0 & -I^{-1}S(\omega_e)I \end{bmatrix} \begin{bmatrix} \sigma_e \\ \omega_e \end{bmatrix} + \begin{bmatrix} 0 \\ I^{-1} \end{bmatrix} u$$

That is:

$$A(x) = \begin{bmatrix} 0 & G(\sigma_e) \\ 0 & -I^{-1}S(\omega_e)I \end{bmatrix}, B(x) = \begin{bmatrix} 0 \\ I^{-1} \end{bmatrix}, x = \begin{bmatrix} \sigma_e \\ \omega_e \end{bmatrix}$$

We simulate for the operating point:

$$\sigma_e \rightarrow [-1, 0, 1]^T, \omega_e \rightarrow [-5; 0; 5]^o/sec$$

After computation, each component of the control gain
matrix $K(x)$ is put in a lookup table (of dimension 3x6)
for a particular attitude and angular velocity error point.
During the operation they are interpolated linearly (or in
any another way chosen by the designer). If the control
gain matrix is denoted as shown below:

$$\mathbf{K}(\mathbf{x}) = \begin{bmatrix} k_{11}(\mathbf{x}) & k_{12}(\mathbf{x}) & k_{13}(\mathbf{x}) & k_{14}(\mathbf{x}) & k_{15}(\mathbf{x}) & k_{16}(\mathbf{x}) \\ k_{21}(\mathbf{x}) & k_{22}(\mathbf{x}) & k_{23}(\mathbf{x}) & k_{24}(\mathbf{x}) & k_{25}(\mathbf{x}) & k_{26}(\mathbf{x}) \\ k_{31}(\mathbf{x}) & k_{32}(\mathbf{x}) & k_{33}(\mathbf{x}) & k_{34}(\mathbf{x}) & k_{35}(\mathbf{x}) & k_{36}(\mathbf{x}) \end{bmatrix}$$

, the final control law (required actuation):

$$\tau_a = -K(x)x + S(R(\sigma_e)\omega_{io}^o)I\omega_e + S(\omega_e)IR(\sigma_e)\omega_{io}^o$$

$$+S(R(\sigma_e)\omega_{io}^o)IR(\sigma_e)\omega_{io}^o + IS(\omega_e)R(\sigma_e)\omega_{io}^o$$

6. SIMULATION OUTLINE

The paper performs an example simulation. The pa-
rameters provided have been taken to re-create the results,

such as the Inertia Matrix, initial attitude, and the Quadratic Performance Indices (parameters for Q and R).

$$I = \begin{bmatrix} 9.8194 & -0.0721 & -0.2893 \\ -0.0721 & 9.7030 & -0.1011 \\ -0.2893 & -0.1011 & 9.7309 \end{bmatrix}, \sigma_{initial} = [0.33 \ -0.33 \ -0.33]^T$$

i.e. The satellite starts from Euler angles- $\Phi = [90^\circ \ 0^\circ \ -90^\circ]$ and moves to the origin. The control law picked tracks the state vector of the satellite to 0 since r is taken as 0 in $u = -K(x)x + r$.

Other parameters like the orbital velocity $\omega_{io}^o = 0.0011 \text{ rad/sec}$ (taken for a LEO satellite in a circular orbit, assuming no changes, and along the y-axis of the inertial frame), and initial angular velocity error- $\omega_e(initial) = 0$ have been assumed.

The cases of Quadratic Performance Indices and attempts to pick the best ones are shown below:

$$\mathbf{Q}(\mathbf{x}) = \mathbf{Q} = \begin{bmatrix} q_1 \mathbf{I}_{3 \times 3} & 0 \\ 0 & q_2 \mathbf{I}_{3 \times 3} \end{bmatrix}, \mathbf{R}(\mathbf{x}) = \mathbf{R} = r \mathbf{I}_{3 \times 3}$$

In the table above, comments on the simulation results are also given, the comments on the torque levels are given according to the characteristics of the satellite model presented in [Kaplan(2006)] where it is advised not to have a torque requirement larger than 0.02 Nm. However, there is no actuator limit set on the actuator torque control law. The graphical results are reproduced in the next section.

7. GRAPHICAL RESULTS

From the figures (at the end of the report), the best candidate for application seems to be the second entry in Table 1. The torque requirement of the fourth case seems lowest, but the system response is too slow and the torque levels of the second case is far under 0.02 Nm, making it the best set of performance indices. Similarly, the best performance indices for every operating point can be stored and used when required.

8. CONCLUSION

The report covers the design of an optimal attitude controller based on state dependent Riccati equation. The

TABLE 1. The values of quadratic weighting coefficients and comments on the simulation results

Case	q_1	q_2	r	Comments
1	0.1	0.000001	1	$\max(\tau_0) \geq 0.02 \text{ N} \cdot \text{m}$
2	0.1	0.000001	1000	τ_0 normal
3	0.001	0.000001	1	τ_0 normal
4	0.001	0.000001	1000	τ_0 low, too slow

satellite attitude kinematics is represented by the Modified Rodriguez Parameters (MRP). The analytical solution for the control law is difficult, hence certain points are taken in the operating range and a lookup table is created of the necessary control gains evaluated at those points and are interpolated throughout the operation of the satellite to form point wise stable control laws.

The entire process, from linearized modeling at operating points, finding controller gains by solving Riccati through the care() tool, to solving the model through ode45 and plotting graphs, was coded on MATLAB. The simulations show that the attitude control laws operate successfully. Improvements to this paper include analyzing global stability of the closed loop controller.

ACKNOWLEDGMENT

Thanks goes to Prof. Luoyi Tao for his guidance and support.

REFERENCES

- [1] Wikipedia, *Rotation formalisms in three dimensions*
- [2] R. Ozgur Doruk, "*SDRE Based Attitude Control Using Modified Rodriguez Parameters*", Middle East Technical University, Kalkanli, Guzelyurt, Mersin 10, TURKEY.
- [3] Kaplan C., "*Leo Satellites: Attitude Determination And Control Components ; Some Linear Attitude Control Techniques*", M.Sc Thesis, METU 2006
- [4] Cimen T., "*State dependent Riccati equation (SDRE) control: a survey*", Proceedings of the 17th World Congress of IFAC, Seoul, Korea, 2008

9. PLOTS

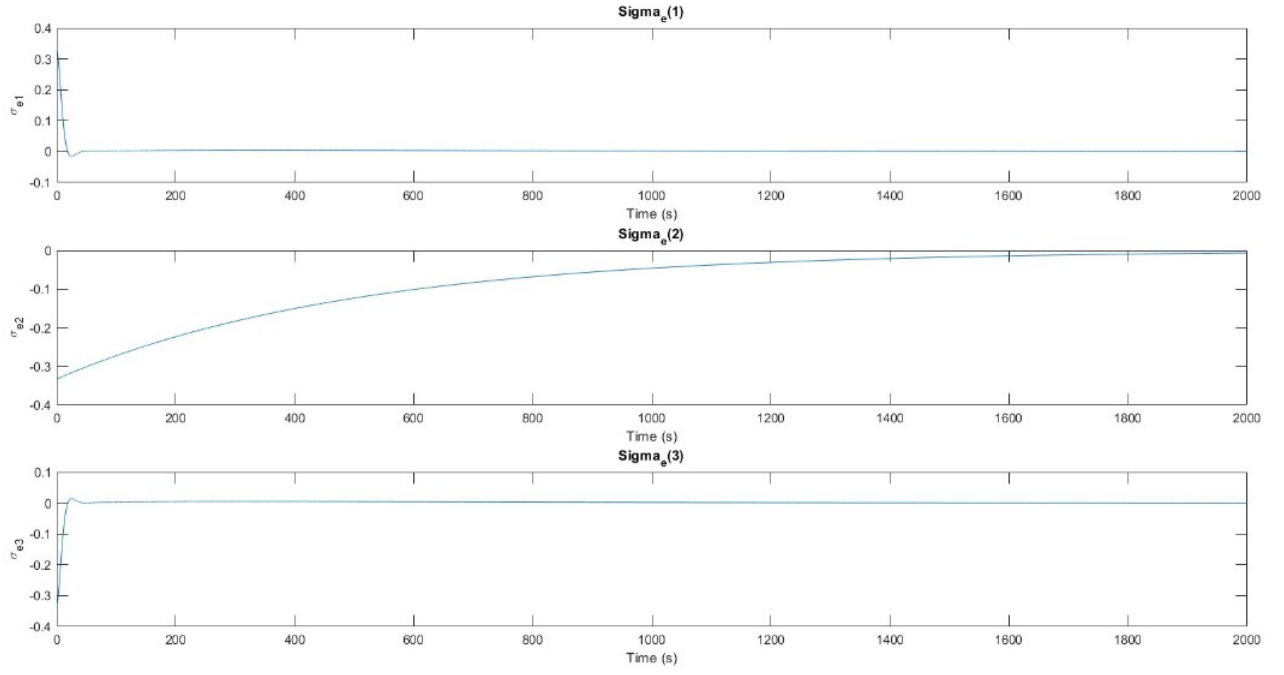


Figure 1: Attitude Error for Case 1

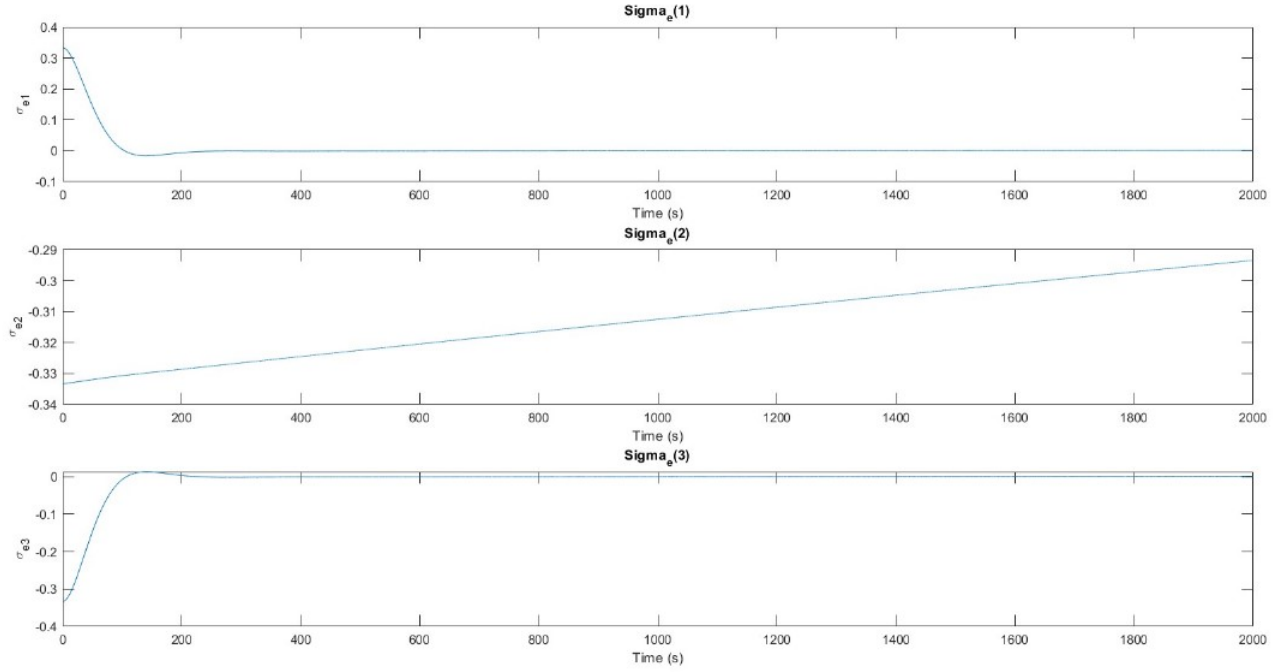


Figure 2: Attitude Error for Case 2

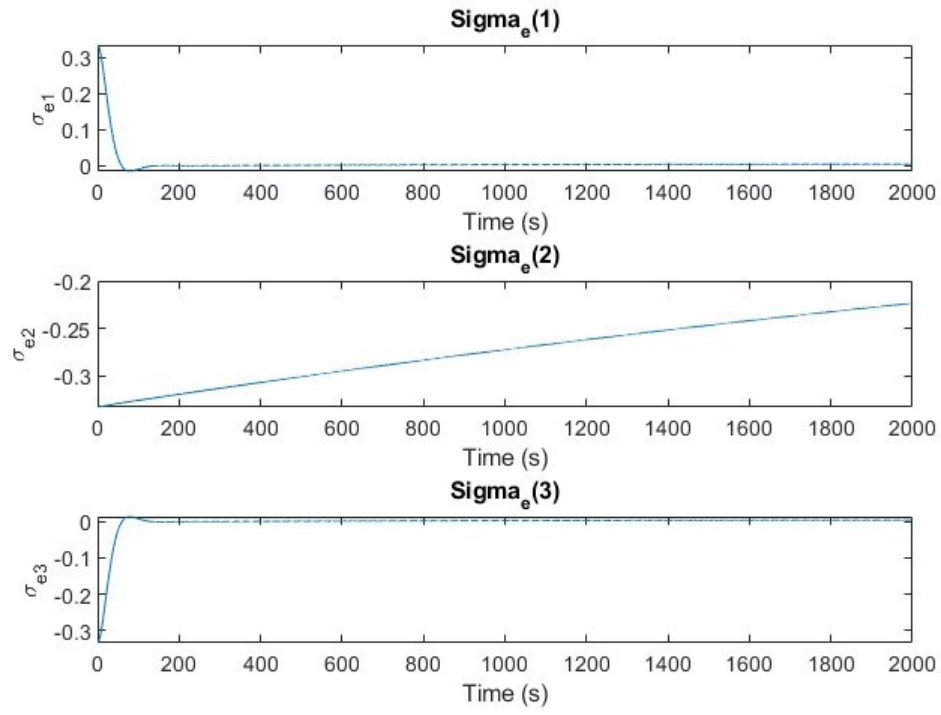


Figure 3: Attitude Error for Case 3

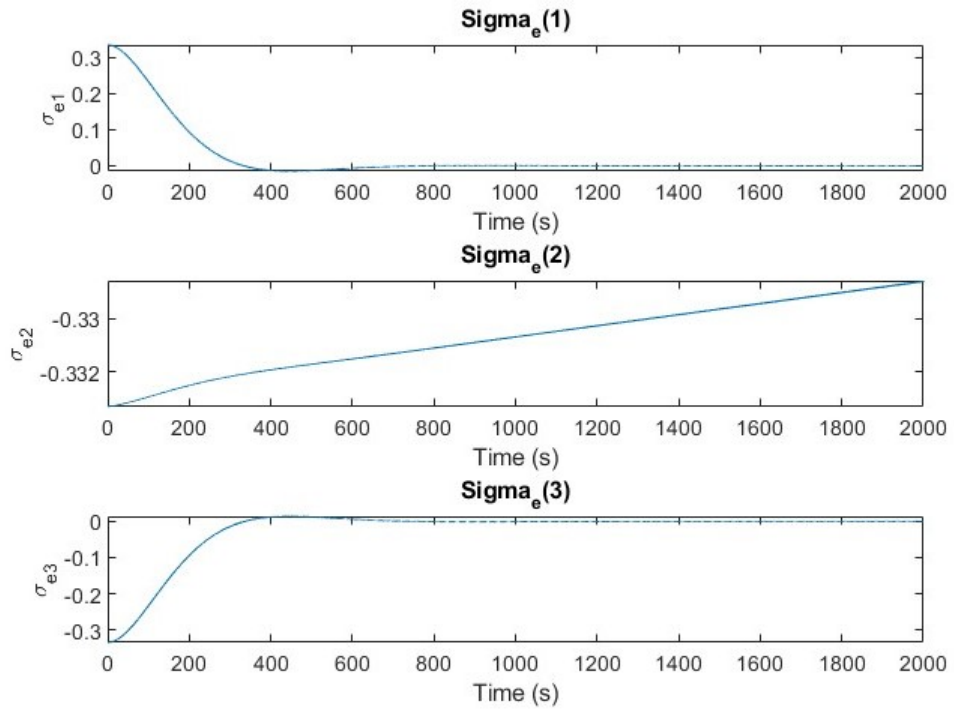


Figure 4: Attitude Error for Case 4

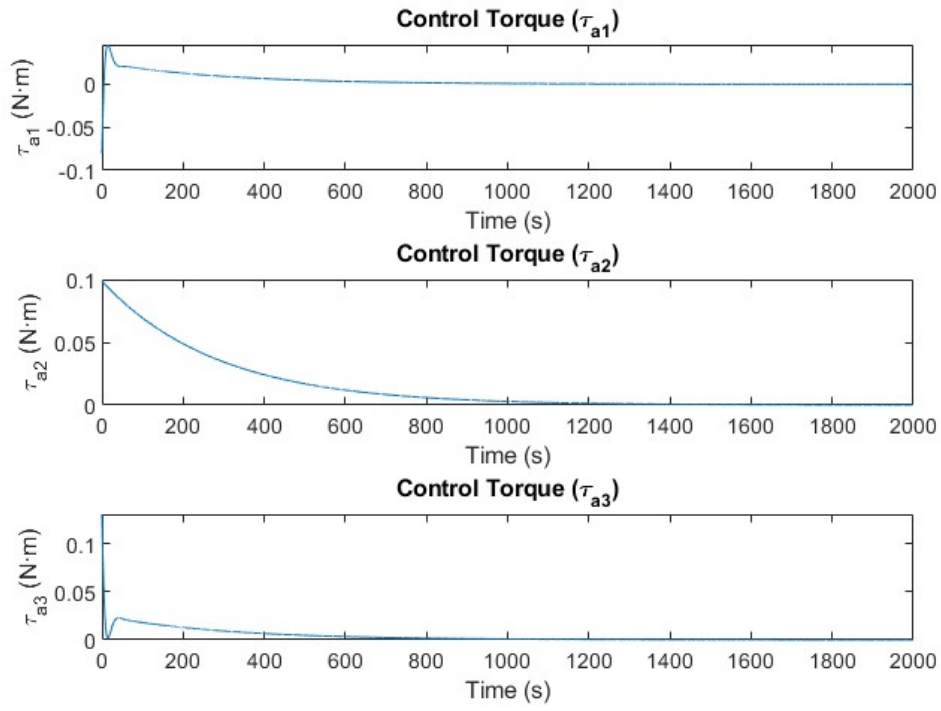


Figure 5: Torque Requirement for Case 1

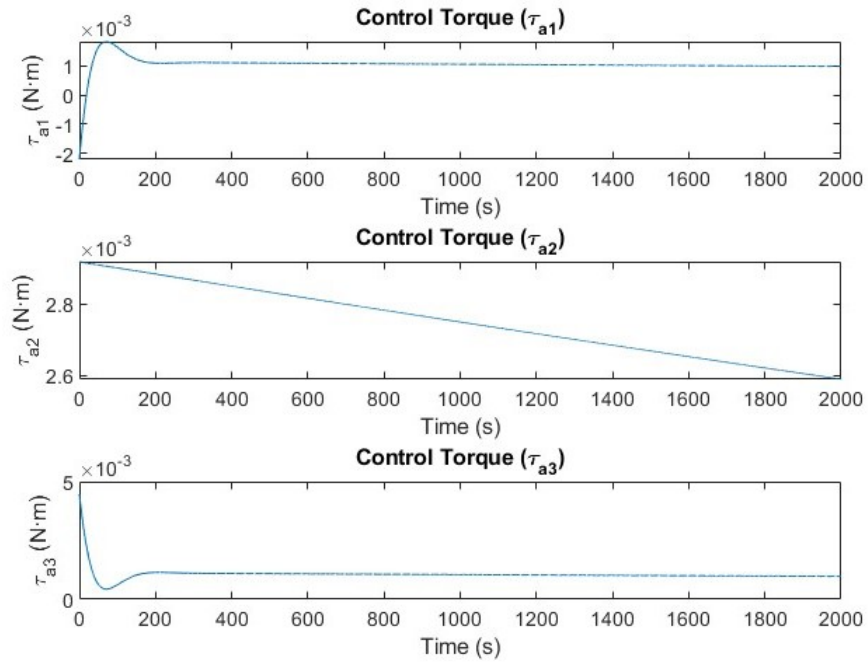


Figure 6: Torque Requirement for Case 2

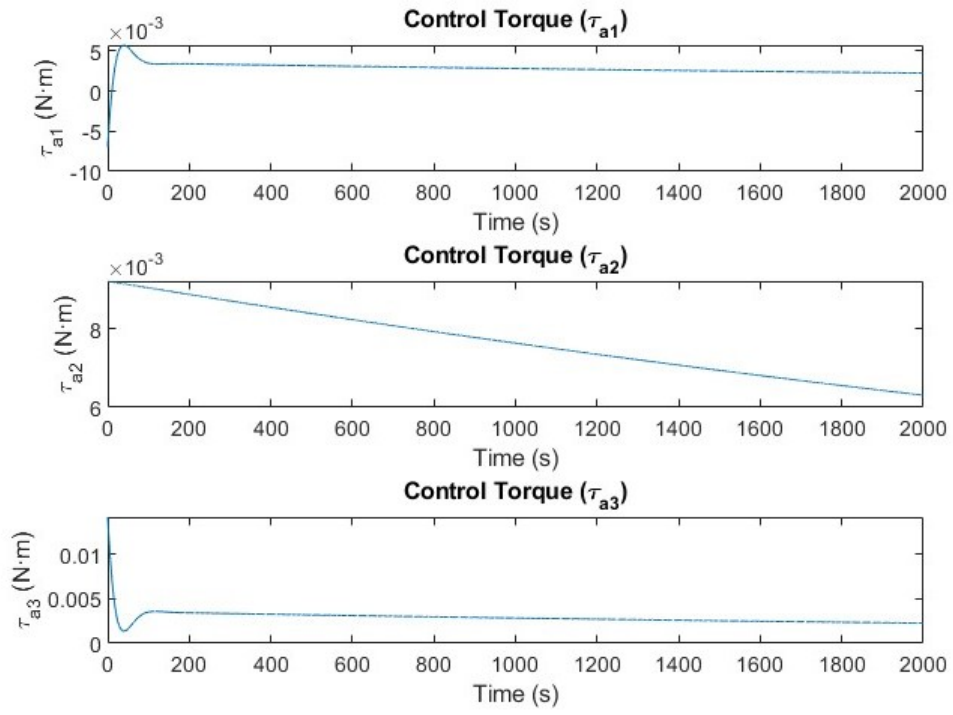


Figure 7: Torque Requirement for Case 3

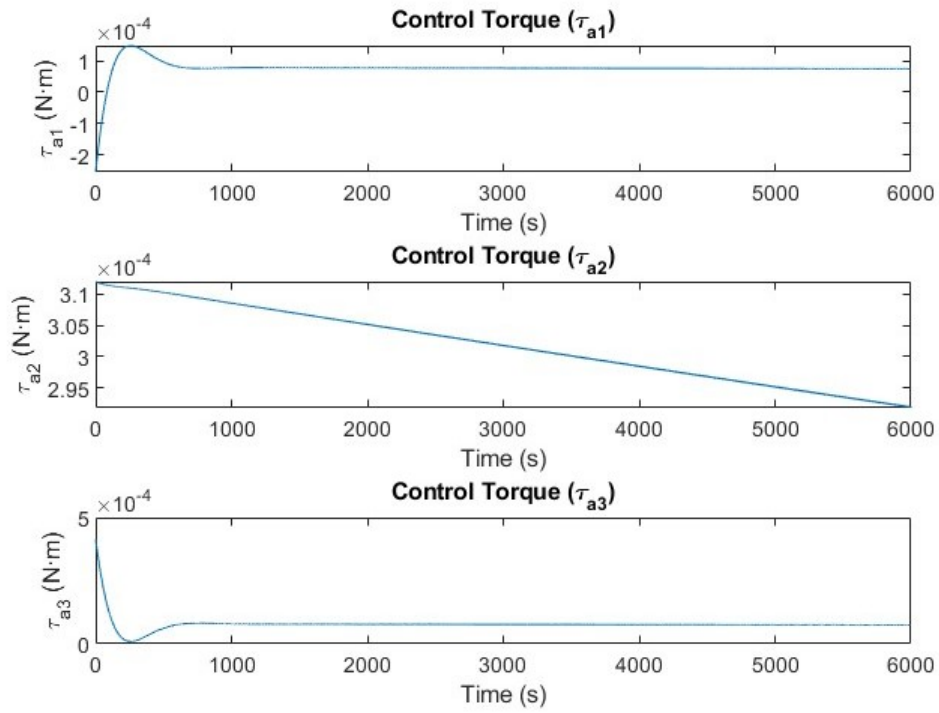


Figure 8: Torque Requirement for Case 4

MATLAB Code

```
% =====
% Satellite Attitude Control Simulation
% =====
% Based on Kaplan (2006)

% Inertia Matrix (from Kaplan, 2006)
I = [9.8194 -0.0721 -0.2893;
     -0.0721 9.7030 -0.1011;
     -0.2893 -0.1011 9.7309];

% =====
% Helper Functions
% =====

% Cross operator matrix
S = @(v) [ 0,  -v(3), v(2);
          v(3), 0,  -v(1);
          -v(2), v(1), 0  ];

% G matrix
G = @(v) 0.5 * ((1 - v.' * v) / 2 * eye(3) + S(v) + (v * v.'));

% Rotation matrix R derived from attitude error
R_transform = @(sigma_e) eye(3) + ...
    (8 / (1 + sigma_e' * sigma_e)^2) * (S(sigma_e))^2 + ...
    (4 * (1 - sigma_e' * sigma_e) / (1 + sigma_e' * sigma_e)^2) * S(sigma_e);

% =====
% Operating Point and Linearization
% =====

% Operating point values
sigma_op = [-1; 0; 1];
omega_op = [-5; 0; 5];

% State-space matrices (linearized at operating point)
A = [zeros(3), G(sigma_op);
     zeros(3), -inv(I) * S(omega_op) * I];
B = [zeros(3); inv(I)];

% =====
% Quadratic Cost Matrices for LQR
```

```

% =====

q1 = 0.1; q2 = 1e-6; % Performance indices
Q = [q1 * eye(3), zeros(3); zeros(3), q2 * eye(3)]; % State weighting matrix
R = eye(3); % Control weighting matrix

% Solve Riccati equation and compute gain K
[P, ~, K] = care(A, B, Q, R);

% =====
% Control Torque Function
% =====

% Orbital velocity (for LEO satellite)
wio = [0; 0; 1.91986e-5]; % rad/s

% Control torque
compute_tau_a = @(x) -K * x + ...
    S(R_transform(x(1:3)) * wio) * I * x(4:6) + ...
    S(x(4:6)) * I * R_transform(x(1:3)) * wio + ...
    S(R_transform(x(1:3)) * wio) * I * R_transform(x(1:3)) * wio + ...
    I * S(x(4:6)) * R_transform(x(1:3)) * wio;

% =====
% State Dynamics and Simulation
% =====

% State dynamics (linearized)
state_dynamics = @(t, x) (A - B * K) * x;

% Time span for simulation
tspan = [0, 2000]; % Time in seconds

% Initial conditions
sigma_e0 = [0.3333; -0.3333; -0.3333]; % Initial attitude error
omega_e0 = [0; 0; 0]; % Initial angular velocity
x0 = [sigma_e0; omega_e0]; % Combined initial state

% Solve the ODE for state evolution
[t, x] = ode45(state_dynamics, tspan, x0);

% Extract sigma_e (attitude error) and omega_e (angular velocity)
sigma_e = x(:, 1:3);
omega_e = x(:, 4:6);

```

```

% Compute control torques over time
tau_a = zeros(length(t), 3);
for i = 1:length(t)
    tau_a(i, :) = compute_tau_a(x(i, :));
end

% =====
% Plotting Results
% =====

% Control torques
figure;
subplot(3, 1, 1);
plot(t, tau_a(:, 1), 'r', 'LineWidth', 1.5); % Tau_a1 in red
title('Control Torque ( $\tau_{a1}$ )');
xlabel('Time (s)');
ylabel('\tau_{a1} (N·m)');

subplot(3, 1, 2);
plot(t, tau_a(:, 2), 'g', 'LineWidth', 1.5); % Tau_a2 in green
title('Control Torque ( $\tau_{a2}$ )');
xlabel('Time (s)');
ylabel('\tau_{a2} (N·m)');

subplot(3, 1, 3);
plot(t, tau_a(:, 3), 'b', 'LineWidth', 1.5); % Tau_a3 in blue
title('Control Torque ( $\tau_{a3}$ )');
xlabel('Time (s)');
ylabel('\tau_{a3} (N·m)');

% Optional: Add grid and formatting
for i = 1:3
    subplot(3, 1, i);
    grid on;
end

% Save plots for report use
saveas(gcf, 'control_torques.png'); % Save as PNG

```